Experiment 'p_14_2_selection_sort_interval_array_RERUN-1' Results

June 6, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_14_2_selection_sort_interval_array_RERUN-1
Natural language statement: Write a method that implements the selection sort algorithm to sort an array of values from a finite interval.
Method signature: p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (sorted: array<int>)

Ensures

- `forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]`
- `multiset(inputs[..]) == multiset(sorted[..])`

Requires

- `forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method that implements the selection sort algorithm to sort an array of values
    ↪ from a finite interval.

The signature should be:

p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (sorted:
    ↪ array<int>)

The method should respect the following contract:

requires forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound, ensures forall i
    ↪ , j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j], ensures multiset(
    ↪ inputs[..]) == multiset(sorted[..])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnkN9Z9GJq0HM0zbiz3U3bJJtu59p
Model: gpt-5.5-2026-04-23
Created at: 1780748699
Finish reason: stop
Usage: 380 tokens in, and 10636 tokens out

```
{
  var n := inputs.Length;
  sorted := new int[n];

  var c := 0;
  while c < n
    invariant 0 <= c <= n
    invariant sorted.Length == n
    invariant forall k :: 0 <= k < c ==> sorted[k] == inputs[k]
  {
    sorted[c] := inputs[c];
    c := c + 1;
  }

  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  assert |sorted[..]| == |inputs[..]|;
  forall k | 0 <= k < n
```

```
  ensures sorted[..][k] == inputs[..][k]
{
  assert sorted[..][k] == sorted[k];
  assert inputs[..][k] == inputs[k];
}
assert sorted[..] == inputs[..];

var i := 0;
while i < n
  invariant 0 <= i <= n
  invariant sorted.Length == n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall a, b :: 0 <= a < b < i ==> sorted[a] <= sorted[b]
  invariant forall a, b :: 0 <= a < i <= b < n ==> sorted[a] <= sorted[b]
{
  var min := i;
  var j := i + 1;
  while j < n
    invariant i < n
    invariant i < j <= n
    invariant i <= min < j
    invariant sorted.Length == n
    invariant forall k :: i <= k < j ==> sorted[min] <= sorted[k]
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      assert forall k :: i <= k < j ==> sorted[oldMin] <= sorted[k];
      min := j;
      forall k | i <= k < j
        ensures sorted[min] <= sorted[k]
      {
        assert sorted[oldMin] <= sorted[k];
        assert sorted[min] < sorted[oldMin];
      }
      assert sorted[min] <= sorted[j];
    } else {
      forall k | i <= k < j
        ensures sorted[min] <= sorted[k]
      {
      }
      assert sorted[min] <= sorted[j];
    }
    forall k | i <= k < j + 1
      ensures sorted[min] <= sorted[k]
    {
      if k < j {
        assert sorted[min] <= sorted[k];
      } else {
        assert k == j;
        assert sorted[min] <= sorted[j];
      }
```

```
  }
  j := j + 1;
}

assert j == n;
assert i <= min < n;

ghost var pre := sorted[..];
assert forall a, b :: 0 <= a < b < i ==> pre[a] <= pre[b];
assert forall a, b :: 0 <= a < i <= b < n ==> pre[a] <= pre[b];
assert forall k :: i <= k < n ==> pre[min] <= pre[k];

if min != i {
  assert i < min;
  sorted[i], sorted[min] := sorted[min], sorted[i];

  assert sorted[i] == pre[min];
  assert sorted[min] == pre[i];
  forall k | 0 <= k < n && k != i && k != min
    ensures sorted[k] == pre[k]
  {
  }

  ghost var afterSwap := pre[..i] + [pre[min]] + pre[i + 1..min] + [pre[i]] + pre[min
      ↪ + 1..];
  assert |afterSwap| == n;
  forall k | 0 <= k < n
    ensures sorted[..][k] == afterSwap[k]
  {
    assert sorted[..][k] == sorted[k];
    if k < i {
      assert sorted[k] == pre[k];
      assert afterSwap[k] == pre[k];
    } else if k == i {
      assert sorted[k] == pre[min];
      assert afterSwap[k] == pre[min];
    } else if k < min {
      assert sorted[k] == pre[k];
      assert afterSwap[k] == pre[k];
    } else if k == min {
      assert sorted[k] == pre[i];
      assert afterSwap[k] == pre[i];
    } else {
      assert min < k;
      assert sorted[k] == pre[k];
      assert afterSwap[k] == pre[k];
    }
  }
  assert sorted[..] == afterSwap;
  assert pre == pre[..i] + [pre[i]] + pre[i + 1..min] + [pre[min]] + pre[min + 1..];
  calc {
```

```
    multiset(sorted[..]);
    ==
    multiset(afterSwap);
    ==
    multiset(pre[..i] + [pre[min]] + pre[i + 1..min] + [pre[i]] + pre[min + 1..]);
    ==
    multiset(pre[..i]) + multiset([pre[min]]) + multiset(pre[i + 1..min]) + multiset([
        ↪ pre[i]]) + multiset(pre[min + 1..]);
    ==
    multiset(pre[..i]) + multiset([pre[i]]) + multiset(pre[i + 1..min]) + multiset([
        ↪ pre[min]]) + multiset(pre[min + 1..]);
    ==
    multiset(pre[..i] + [pre[i]] + pre[i + 1..min] + [pre[min]] + pre[min + 1..]);
    ==
    multiset(pre);
  }
} else {
  assert sorted[..] == pre;
  assert sorted[i] == pre[min];
  assert sorted[min] == pre[i];
  forall k | 0 <= k < n && k != i && k != min
    ensures sorted[k] == pre[k]
  {
  }
  assert multiset(sorted[..]) == multiset(pre);
}

assert sorted[i] == pre[min];
assert sorted[min] == pre[i];
assert forall k :: 0 <= k < n && k != i && k != min ==> sorted[k] == pre[k];
assert multiset(sorted[..]) == multiset(inputs[..]);

forall a, b | 0 <= a < b < i + 1
  ensures sorted[a] <= sorted[b]
{
  if b < i {
    assert a != i && b != i;
    assert a != min && b != min;
    assert sorted[a] == pre[a];
    assert sorted[b] == pre[b];
    assert pre[a] <= pre[b];
  } else {
    assert b == i;
    assert a < i;
    assert a != i;
    assert a != min;
    assert sorted[a] == pre[a];
    assert sorted[b] == pre[min];
    assert pre[a] <= pre[min];
  }
}
```

```
    forall a, b | 0 <= a < i + 1 <= b < n
      ensures sorted[a] <= sorted[b]
    {
      assert b != i;
      if a < i {
        assert a != i;
        assert a != min;
        assert sorted[a] == pre[a];
        if b == min {
          assert sorted[b] == pre[i];
          assert pre[a] <= pre[i];
        } else {
          assert sorted[b] == pre[b];
          assert pre[a] <= pre[b];
        }
      } else {
        assert a == i;
        assert sorted[a] == pre[min];
        if b == min {
          assert sorted[b] == pre[i];
          assert pre[min] <= pre[i];
        } else {
          assert sorted[b] == pre[b];
          assert pre[min] <= pre[b];
        }
      }
    }

    i := i + 1;
  }

  assert i == n;
  forall a, b | 0 <= a < b < sorted.Length
    ensures sorted[a] <= sorted[b]
  {
    assert b < i;
  }
  assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Final Program

```
method p_14_2_selection_sort_interval_array(inputs: array<int>, bound: nat) returns (
    ↪ sorted: array<int>)
      requires forall i :: 0 <= i < inputs.Length ==> 0 <= inputs[i] <= bound
      ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] <= sorted[j]
      ensures multiset(inputs[..]) == multiset(sorted[..])
{
  var n := inputs.Length;
```

```
sorted := new int[n];

var c := 0;
while c < n
  invariant 0 <= c <= n
  invariant sorted.Length == n
  invariant forall k :: 0 <= k < c ==> sorted[k] == inputs[k]
{
  sorted[c] := inputs[c];
  c := c + 1;
}

assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
assert |sorted[..]| == |inputs[..]|;
forall k | 0 <= k < n
  ensures sorted[..][k] == inputs[..][k]
{
  assert sorted[..][k] == sorted[k];
  assert inputs[..][k] == inputs[k];
}
assert sorted[..] == inputs[..];

var i := 0;
while i < n
  invariant 0 <= i <= n
  invariant sorted.Length == n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall a, b :: 0 <= a < b < i ==> sorted[a] <= sorted[b]
  invariant forall a, b :: 0 <= a < i <= b < n ==> sorted[a] <= sorted[b]
{
  var min := i;
  var j := i + 1;
  while j < n
    invariant i < n
    invariant i < j <= n
    invariant i <= min < j
    invariant sorted.Length == n
    invariant forall k :: i <= k < j ==> sorted[min] <= sorted[k]
  {
    if sorted[j] < sorted[min] {
      var oldMin := min;
      assert forall k :: i <= k < j ==> sorted[oldMin] <= sorted[k];
      min := j;
      forall k | i <= k < j
        ensures sorted[min] <= sorted[k]
      {
        assert sorted[oldMin] <= sorted[k];
        assert sorted[min] < sorted[oldMin];
      }
      assert sorted[min] <= sorted[j];
    } else {
```

```
    forall k | i <= k < j
      ensures sorted[min] <= sorted[k]
    {
    }
    assert sorted[min] <= sorted[j];
  }
  forall k | i <= k < j + 1
    ensures sorted[min] <= sorted[k]
  {
    if k < j {
      assert sorted[min] <= sorted[k];
    } else {
      assert k == j;
      assert sorted[min] <= sorted[j];
    }
  }
  j := j + 1;
}

assert j == n;
assert i <= min < n;

ghost var pre := sorted[..];
assert forall a, b :: 0 <= a < b < i ==> pre[a] <= pre[b];
assert forall a, b :: 0 <= a < i <= b < n ==> pre[a] <= pre[b];
assert forall k :: i <= k < n ==> pre[min] <= pre[k];

if min != i {
  assert i < min;
  sorted[i], sorted[min] := sorted[min], sorted[i];

  assert sorted[i] == pre[min];
  assert sorted[min] == pre[i];
  forall k | 0 <= k < n && k != i && k != min
    ensures sorted[k] == pre[k]
  {
  }

  ghost var afterSwap := pre[..i] + [pre[min]] + pre[i + 1..min] + [pre[i]] + pre[min
      ↪ + 1..];
  assert |afterSwap| == n;
  forall k | 0 <= k < n
    ensures sorted[..][k] == afterSwap[k]
  {
    assert sorted[..][k] == sorted[k];
    if k < i {
      assert sorted[k] == pre[k];
      assert afterSwap[k] == pre[k];
    } else if k == i {
      assert sorted[k] == pre[min];
      assert afterSwap[k] == pre[min];
```

```
    } else if k < min {
      assert sorted[k] == pre[k];
      assert afterSwap[k] == pre[k];
    } else if k == min {
      assert sorted[k] == pre[i];
      assert afterSwap[k] == pre[i];
    } else {
      assert min < k;
      assert sorted[k] == pre[k];
      assert afterSwap[k] == pre[k];
    }
  }
  assert sorted[..] == afterSwap;
  assert pre == pre[..i] + [pre[i]] + pre[i + 1..min] + [pre[min]] + pre[min + 1..];
  calc {
    multiset(sorted[..]);
    ==
    multiset(afterSwap);
    ==
    multiset(pre[..i] + [pre[min]] + pre[i + 1..min] + [pre[i]] + pre[min + 1..]);
    ==
    multiset(pre[..i]) + multiset([pre[min]]) + multiset(pre[i + 1..min]) + multiset([
        ↪ pre[i]]) + multiset(pre[min + 1..]);
    ==
    multiset(pre[..i]) + multiset([pre[i]]) + multiset(pre[i + 1..min]) + multiset([
        ↪ pre[min]]) + multiset(pre[min + 1..]);
    ==
    multiset(pre[..i] + [pre[i]] + pre[i + 1..min] + [pre[min]] + pre[min + 1..]);
    ==
    multiset(pre);
  }
} else {
  assert sorted[..] == pre;
  assert sorted[i] == pre[min];
  assert sorted[min] == pre[i];
  forall k | 0 <= k < n && k != i && k != min
    ensures sorted[k] == pre[k]
  {
  }
  assert multiset(sorted[..]) == multiset(pre);
}

assert sorted[i] == pre[min];
assert sorted[min] == pre[i];
assert forall k :: 0 <= k < n && k != i && k != min ==> sorted[k] == pre[k];
assert multiset(sorted[..]) == multiset(inputs[..]);

forall a, b | 0 <= a < b < i + 1
  ensures sorted[a] <= sorted[b]
{
  if b < i {
```

```
      assert a != i && b != i;
      assert a != min && b != min;
      assert sorted[a] == pre[a];
      assert sorted[b] == pre[b];
      assert pre[a] <= pre[b];
    } else {
      assert b == i;
      assert a < i;
      assert a != i;
      assert a != min;
      assert sorted[a] == pre[a];
      assert sorted[b] == pre[min];
      assert pre[a] <= pre[min];
    }
  }

  forall a, b | 0 <= a < i + 1 <= b < n
    ensures sorted[a] <= sorted[b]
  {
    assert b != i;
    if a < i {
      assert a != i;
      assert a != min;
      assert sorted[a] == pre[a];
      if b == min {
        assert sorted[b] == pre[i];
        assert pre[a] <= pre[i];
      } else {
        assert sorted[b] == pre[b];
        assert pre[a] <= pre[b];
      }
    } else {
      assert a == i;
      assert sorted[a] == pre[min];
      if b == min {
        assert sorted[b] == pre[i];
        assert pre[min] <= pre[i];
      } else {
        assert sorted[b] == pre[b];
        assert pre[min] <= pre[b];
      }
    }
  }

  i := i + 1;
}

assert i == n;
forall a, b | 0 <= a < b < sorted.Length
  ensures sorted[a] <= sorted[b]
{
```

```
    assert b < i;
  }
  assert multiset(inputs[..]) == multiset(sorted[..]);
}
```

Total Token Usage

Input tokens: 380
Output tokens: 10636
Reasoning tokens: 8590
Sum of 'total tokens': 11016

Experiment Timings

Overall Experiment started at 1780748698609, ended at 1780748899870, lasting 201261ms (201.26 seconds)
Iteration #1 started at 1780748698612, ended at 1780748899870, lasting 201258ms (201.26 seconds)